

Sección 1 - Introducción a Redis

Redis (Remote Dictionary Server) es una base de datos NoSQL en memoria, de tipo clave-valor, ampliamente utilizada como caché, broker de mensajes y motor de estructuras de datos en tiempo real. Responder las siguientes preguntas antes de comenzar con los ejercicios prácticos.

1. ¿Qué tipo de base de datos es Redis? ¿En qué se diferencia de una base de datos relacional y de otras bases de datos NoSQL como MongoDB?

Redis es una base de datos NoSQL clave-valor en memoria

A diferencia de una relacional, no usa tablas, filas ni SQL Tradicional

Frente a MongoDB, Redis no se centra en documentos persistentes, sino en estructuras rápidas en memoria como strings, listas, sets, hashes, sorted sets

2. ¿Dónde almacena los datos Redis? ¿Qué implicancias tiene esto en términos de velocidad y de persistencia?

Redis almacena los datos en memoria RAM

Esto lo hace muy rápido pero implica que la persistencia debe configurarse aparte para no perder datos ante reinicios o fallos

3. ¿Qué tipos de datos soporta Redis? Listar y describir brevemente cada uno.

- Strings
 - Valores simples de texto, números o binarios
- Lists
 - Listas ordenadas
- Sets
 - Conjuntos sin repetidos
- Sorted Sets
 - Conjuntos ordenados
- Hashes
 - Mapas campo-valor
- Streams
 - Secuencias de eventos
- Bitmaps
 - Operaciones sobre bits
- HyperLogLogs
 - Conteo aproximado de elementos únicos
- Geospatial

- Coordenadas geográficas basadas en sorted sets

4. Enunciar las características principales de Redis.

- Alta velocidad
- Almacenamiento en memoria
- Estructura clave-valor
- Soporte de múltiples tipos de datos
- Expiración de claves
- Replicación
- Clustering

5. Comparar Redis con los RDBMS: ¿en qué casos conviene usar Redis en lugar de una base de datos relacional y en cuáles no?

Conviene usar redis cuando se necesita mucha velocidad: caché, sesiones, colas simples, contadores, rate limiting o datos temporales

No conviene como reemplazo principal de un RDBMS cuando se requieren consultas complejas, relaciones fuertes, integridad referencial o joins

6. ¿Redis tiene soporte para transacciones? ¿Cómo funcionan? ¿Qué garantías ofrecen y qué limitaciones tienen respecto de las transacciones ACID?

Sí, se soportan transacciones con `MULTI`, `EXEC`, `DISCARD` y `WATCH`

Las operaciones se encolan y luego se ejecutan juntas

Garantiza ejecución secuencial y atómica del bloque, pero no ofrece rollback automático como una base de datos ACID tradicional

7. ¿Redis tiene persistencia? Describir los mecanismos disponibles (RDB y AOF) e indicar las diferencias entre ellos.

Sí, tiene persistencia

- RDB
 - Genera snapshots del estado en determinados momentos
 - Es compacto y rápido para restaurar, pero puede perder cambios recientes
- AOF
 - Guarda cada operación de escritura en un log
 - Permite menor pérdida de datos pero suele ocupar más espacio y ser más lento

8. ¿Cuáles son los principales casos de uso de Redis en aplicaciones reales?

- Caché
- Sesiones
- Colas
- Contadores
- Rate limiting
- Datos temporales con expiración

- Procesamiento de eventos en tiempo real

Sección 2 - Manejo de Strings

2.1 Valores de texto

9. Agregar una clave package con el valor "Bariloche 3 days".

```
127.0.0.1:6379> SET package "Bariloche 3 days"
OK
```

10. Agregar una clave user con el valor "Turismo BD2". Obtener el valor de la clave user.

```
127.0.0.1:6379> SET user "Turismo BD2"
OK
127.0.0.1:6379> GET user
"Turismo BD2"
```

11. Obtener todas las claves almacenadas actualmente.

```
127.0.0.1:6379> KEYS *
1) "package"
2) "user"
```

12. Agregar una clave user con el valor "Cronos Turismo". ¿Cuál es el valor actual de la clave user?

```
127.0.0.1:6379> SET user "Cronos Turismo"
OK
127.0.0.1:6379> GET user
"Cronos Turismo"
```

13. Concatenar " S.A." a la clave user. ¿Cuál es el valor actual de la clave user?

```
127.0.0.1:6379> APPEND user " S.A."
(integer) 19
127.0.0.1:6379> GET user
"Cronos Turismo S.A."
```

14. Eliminar la clave user. ¿Qué valor retorna si se intenta obtener la clave user luego de eliminarla?

```
127.0.0.1:6379> DEL user
(integer) 1
127.0.0.1:6379> GET user
(nil)
```

2.2 Valores numericos

15. Verificar si existe la clave visits.

```
127.0.0.1:6379> EXISTS visits  
(integer) 0
```

16. Agregar una clave visits con el valor 0.

```
127.0.0.1:6379> SET visits 0  
OK
```

17. Incrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
127.0.0.1:6379> GET visits  
"0"  
127.0.0.1:6379> INCR visits  
(integer) 1  
127.0.0.1:6379> GET visits  
"1"
```

18. Incrementar en 5 la clave visits. ¿Cuál es el valor actual?

```
127.0.0.1:6379> INCRBY visits 5  
(integer) 6  
127.0.0.1:6379> GET visits  
"6"
```

19. Decrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
127.0.0.1:6379> DECR visits  
(integer) 5  
127.0.0.1:6379> GET visits  
"5"
```

20. Incrementar en 2 la clave visits. ¿Cuál es el valor actual?

```
127.0.0.1:6379> INCRBY visits 2  
(integer) 7  
127.0.0.1:6379> GET visits  
"7"
```

21. Agregar una clave "value package" con el valor 539789.32.

```
127.0.0.1:6379> SET "value package" 539789.32
OK
```

22. Incrementar en 20000 la clave "value package". ¿Cuál es el valor actual?

```
127.0.0.1:6379> INCRBYFLOAT "value package" 20000
"559789.3199999999999999318"
127.0.0.1:6379> GET "value package"
"559789.3199999999999999318"
```

23. ¿Cual es el tipo de datos de "value package", visits y user?

```
127.0.0.1:6379> TYPE "value package"
string
127.0.0.1:6379> type visits
string
127.0.0.1:6379> TYPE user
none
```

Sección 3 - Manejo de Claves

24. Obtener todas las claves que empiecen con "v".

```
127.0.0.1:6379> KEYS v*
1) "value package"
2) "visits"
```

25. Obtener todas las claves que contengan la letra "t".

```
127.0.0.1:6379> KEYS *t*
1) "visits"
```

26. Obtener todas las claves que terminan con "age".

```
127.0.0.1:6379> KEYS *age
1) "value package"
2) "package"
```

27. Renombrar la clave "package" por "bariloche package".

```
127.0.0.1:6379> RENAME package "bariloche package"
OK
127.0.0.1:6379> KEYS *age
1) "value package"
2) "bariloche package"
```

28. ¿Qué comando se utiliza para renombrar una clave solo si el nombre destino no existe aún?

RENAMENX

```
127.0.0.1:6379> KEYS *age
1) "value package"
2) "bariloche package"
127.0.0.1:6379> RENAMENX "bariloche package" "value package"
(integer) 0
127.0.0.1:6379> KEYS *age
1) "value package"
2) "bariloche package"
127.0.0.1:6379> RENAMENX "bariloche package" "nuevo nombre"
(integer) 1
127.0.0.1:6379> KEYS *age
1) "value package"
```

29. Eliminar todas las claves.

Para borrar todas las claves de la DB actual es `FLUSHDB`, para borrar todas las claves de todas las bases de datos del servidor de redis, se usa `FLUSHALL`

```
127.0.0.1:6379> FLUSHDB
OK
127.0.0.1:6379> KEYS *
(empty array)
```

Sección 4 - Expiración de Claves

30. Agregar una clave `agency` con el valor "Cronos Tours".

```
127.0.0.1:6379> SET agency "Cronos Tours"
OK
```

31. ¿Cuál es el tiempo de vida (TTL) de la clave `agency`?

```
127.0.0.1:6379> TTL agency
(integer) -1
```

32. Agregar una expiración de 30 segundos a la clave `agency`.

```
127.0.0.1:6379> EXPIRE agency 30
(integer) 1
```

33. ¿Cuál es el tiempo de vida de la clave `agency` luego de agregar la expiración?

```
127.0.0.1:6379> TTL agency
(integer) 25
```

34. Pasados los 30 segundos: ¿cuál es el TTL de `agency`? ¿Que retorna si se solicita el valor de `agency`?

```
127.0.0.1:6379> TTL agency  
(integer) -2
```

```
127.0.0.1:6379> GET agency  
(nil)
```

35. Agregar una clave agency con el valor "Cronos Tours" que expire en 20 segundos desde su creación.

```
127.0.0.1:6379> SET agency "Cronos Tours" EX 20  
OK
```

Sección 5 - Listas

36. Insertar una lista llamada pets con el valor "dog".

```
127.0.0.1:6379> RPUSH pets dog  
(integer) 1
```

37. ¿Qué sucede si se ejecuta el comando GET sobre pets? ¿Cómo se obtienen los valores de una lista?

```
127.0.0.1:6379> GET pets  
(error) WRONGTYPE Operation against a key holding the wrong kind of value
```

Para obtener los valores de una lista se usa `LRANGE` y se define un rango de índices (-1 referencia a la última posición)

```
127.0.0.1:6379> LRANGE pets 0 -1  
1) "dog"
```

Para obtener un valor en particular, conociendo el índice, se usa `LINDEX`

```
127.0.0.1:6379> LINDEX pets 0  
"dog"
```

38. Agregar a la lista pets el valor "cat" por la izquierda.

```
127.0.0.1:6379> LPUSH pets cat  
(integer) 2  
127.0.0.1:6379> LRANGE pets 0 -1  
1) "cat"  
2) "dog"
```

39. Agregar a la lista pets el valor "fish" por la derecha.

```
127.0.0.1:6379> RPUSH pets fish  
(integer) 3  
127.0.0.1:6379> LRANGE pets 0 -1  
1) "cat"  
2) "dog"  
3) "fish"
```

40. ¿Qué tipo de dato es el valor de pets?

```
127.0.0.1:6379> TYPE pets  
list
```

41. Eliminar el valor del extremo izquierdo de la lista.

```
127.0.0.1:6379> LPOP pets  
"cat"  
127.0.0.1:6379> LRANGE pets 0 -1  
1) "dog"  
2) "fish"
```

42. Eliminar el valor del extremo derecho de la lista.

```
127.0.0.1:6379> RPOP pets  
"fish"  
127.0.0.1:6379> LRANGE pets 0 -1  
1) "dog"
```

43. Agregar a una clave "vuelo:ar389" los valores: aep, mdz, brc, nqn y mdq.

```
127.0.0.1:6379> RPUSH vuelo:ar389 aep mdz brc nqn mdq  
(integer) 5  
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1  
1) "aep"  
2) "mdz"  
3) "brc"  
4) "nqn"  
5) "mdq"
```

44. Ordenar los valores de la lista "vuelo:ar389". ¿Qué sucede si se solicitan todos los valores de la lista luego de ordenarla?

ALPHA indica que se ordenen alfanuméricamente

```
127.0.0.1:6379> SORT vuelo:ar389 ALPHA  
1) "aep"  
2) "brc"  
3) "mdq"  
4) "mdz"  
5) "nqn"  
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1  
1) "aep"  
2) "mdz"  
3) "brc"  
4) "nqn"  
5) "mdq"
```

SORT devuelve la lista ordenada, pero no modifica la lista original

45. Insertar el valor "fte" inmediatamente después de "brc".


```
127.0.0.1:6379> LINSERT vuelo:ar389 AFTER brc fte
(integer) 6
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "fte"
5) "nqn"
6) "mdq"
```

46. Insertar el valor "ush" inmediatamente antes de "fte".

```
127.0.0.1:6379> LINSERT vuelo:ar389 BEFORE fte ush
(integer) 7
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "ush"
5) "fte"
6) "nqn"
7) "mdq"
```

47. Modificar el último elemento de la lista por "sla".

```
127.0.0.1:6379> LSET vuelo:ar389 -1 sla
OK
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "ush"
5) "fte"
6) "nqn"
7) "sla"
```

48. Obtener la cantidad de elementos de "vuelo:ar389".

```
127.0.0.1:6379> LLEN vuelo:ar389
(integer) 7
```

49. Obtener el tercer valor de "vuelo:ar389".

```
127.0.0.1:6379> LINDEX vuelo:ar389 2
"brc"
```

50. Eliminar el valor "aep" de "vuelo:ar389".

```
127.0.0.1:6379> LREM vuelo:ar389 1 aep
(integer) 1
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "mdz"
2) "brc"
3) "ush"
4) "fte"
5) "nqn"
6) "sla"
```

51. Quedarse únicamente con los valores de las posiciones 3 a 5 de "vuelo:ar389".

Posiciones 3 a 5 como en el comando o el comando debería ser de 2 a 4?

```
127.0.0.1:6379> LTRIM vuelo:ar389 3 5
OK
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "fte"
2) "nqn"
3) "sla"
```

52. Agregar en "vuelo:ar389" el valor "fte". ¿Cuántas veces aparece ahora en la lista?

```
127.0.0.1:6379> LPOS vuelo:ar389 fte COUNT 0
1) (integer) 0
2) (integer) 3
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "fte"
2) "nqn"
3) "sla"
4) "fte"
```

Aparece dos veces, en la posición 1 y en la 4

LPOS permite saber dónde se encuentra un elemento en la lista, con COUNT 0 nos devuelve todas las posiciones encontradas

Sección 6 - Conjuntos (Sets)

53. Agregar un conjunto llamado airports con los siguientes valores:

```
eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juj aep tuc
eqs
```

```
127.0.0.1:6379> SADD airports eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juj aep tuc eqs
(integer) 13
```

```
127.0.0.1:6379> SMEMBERS airports
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "sla"
9) "brc"
10) "cpc"
11) "juj"
12) "tuc"
13) "eqs"
```

54. ¿Cuántos valores tiene el conjunto? ¿Por qué puede diferir de la cantidad de valores ingresados?

```
127.0.0.1:6379> SCARD airports
(integer) 13
```

Puede diferir de la cantidad de valores ingresados porque un SET almacena elementos únicos, con lo cual se eliminan repetidos

55. Listar los valores del conjunto airports.

```
127.0.0.1:6379> SMEMBERS airports
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "sla"
9) "brc"
10) "cpc"
11) "juj"
12) "tuc"
13) "eqs"
```

56. Quitar el valor "cpc" del conjunto airports.

```
127.0.0.1:6379> SREM airports cpc
(integer) 1
127.0.0.1:6379> SMEMBERS airports
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "sla"
9) "brc"
10) "juj"
11) "tuc"
12) "eqs"
```

57. Quitar un valor aleatorio del conjunto airports.

```
127.0.0.1:6379> SPOP airports
"sla"
```

```
127.0.0.1:6379> SMEMBERS airports
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "brc"
9) "juj"
10) "tuc"
11) "eqs"
```

58. ¿Qué cantidad de valores tiene airports ahora?

```
127.0.0.1:6379> SCARD airports
(integer) 11
```

59. Comprobar si "cpc" es miembro del conjunto airports.

```
127.0.0.1:6379> SISMEMBER airports cpc
(integer) 0
```

60. Mover los valores "sla" y "juj" a un nuevo conjunto denominado noa_airports.

"sla" no existe en airports, por lo que no se mueve (indicado con el 0)

```

127.0.0.1:6379> SMOVE airports noa_airports sla
(integer) 0
127.0.0.1:6379> SMOVE airports noa_airports juj
(integer) 1
127.0.0.1:6379> SMEMBERS airports
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "brc"
9) "tuc"
10) "eqs"
127.0.0.1:6379> SMEMBERS noa_airports
1) "juj"

```

61. Obtener la unión de los conjuntos airports y noa_airports. ¿Modifica los conjuntos originales?

```

127.0.0.1:6379> SUNION airports noa_airports
1) "mdz"
2) "mdq"
3) "ush"
4) "aep"
5) "eze"
6) "fte"
7) "juj"
8) "tuc"
9) "nqn"
10) "eqs"
11) "brc"

```

62. Realizar la unión de airports y noa_airports y almacenar el resultado en un nuevo conjunto llamado total_airports.

```

127.0.0.1:6379> SUNIONSTORE total_airports airports noa_airports
(integer) 11

```

```

127.0.0.1:6379> SMEMBERS total_airports
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "brc"
9) "tuc"
10) "eqs"
11) "juj"

```

63. Realizar la intersección entre total_airports y noa_airports.

```
127.0.0.1:6379> SINTER total_airports noa_airports  
1) "juj"
```

64. Realizar la diferencia entre total_airports y noa_airports.

```
127.0.0.1:6379> SDIFF total_airports noa_airports  
1) "brc"  
2) "mdz"  
3) "mdq"  
4) "ush"  
5) "aep"  
6) "eze"  
7) "fte"  
8) "tuc"  
9) "nqn"  
10) "eqs"
```

Sección 7 - Conjuntos Ordenados (Sorted Sets)

65. Agregar a un conjunto ordenado llamado passengers los siguientes datos (score nombre):

```
2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana 2.4 natalia
```

```
127.0.0.1:6379> ZADD passengers 2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana 2.4 natalia  
(integer) 7
```

66. Obtener los valores del conjunto passengers.

```
127.0.0.1:6379> ZRANGE passengers 0 -1  
1) "ivan"  
2) "luciana"  
3) "tomas"  
4) "natalia"  
5) "federico"  
6) "julian"  
7) "alejandra"
```

67. Actualizar el score de luciana a 2.7.

```
127.0.0.1:6379> ZADD passengers 2.7 luciana
(integer) 0
127.0.0.1:6379> ZRANGE passengers 0 -1
1) "ivan"
2) "tomas"
3) "natalia"
4) "federico"
5) "luciana"
6) "julian"
7) "alejandra"
```

68. Agregar al conjunto passengers a silvia con score 5.1.

```
127.0.0.1:6379> ZADD passengers 5.1 silvia
(integer) 1
127.0.0.1:6379> ZRANGE passengers 0 -1
1) "ivan"
2) "tomas"
3) "natalia"
4) "federico"
5) "luciana"
6) "julian"
7) "alejandra"
8) "silvia"
```

69. Incrementar en 2 el score de alejandra.

```
127.0.0.1:6379> ZINCRBY passengers 2 alejandra
"6"
127.0.0.1:6379> ZRANGE passengers 0 -1
1) "ivan"
2) "tomas"
3) "natalia"
4) "federico"
5) "luciana"
6) "julian"
7) "silvia"
8) "alejandra"
```

70. Obtener los valores del conjunto passengers con sus scores.

```
127.0.0.1:6379> ZRANGE passengers 0 -1 WITHSCORES
1) "ivan"
2) "1"
3) "tomas"
4) "2"
5) "natalia"
6) "2.4"
7) "federico"
8) "2.5"
9) "luciana"
10) "2.7"
11) "julian"
12) "3"
13) "silvia"
14) "5.1"
15) "alejandra"
16) "6"
```

71. Obtener los valores del conjunto passengers con sus scores en orden inverso.

```
127.0.0.1:6379> ZREVRANGE passengers 0 -1 WITHSCORES
1) "alejandra"
2) "6"
3) "silvia"
4) "5.1"
5) "julian"
6) "3"
7) "luciana"
8) "2.7"
9) "federico"
10) "2.5"
11) "natalia"
12) "2.4"
13) "tomas"
14) "2"
15) "ivan"
16) "1"
```

72. Obtener la cantidad de elementos del conjunto passengers.

```
127.0.0.1:6379> ZCARD passengers
(integer) 8
```

73. Obtener la cantidad de elementos que tienen scores entre 2 y 3.

```
127.0.0.1:6379> ZCOUNT passengers 2 3
(integer) 5
```

74. Obtener el ranking de julian en el conjunto passengers.

```
127.0.0.1:6379> ZRANK passengers julian
(integer) 5
```


75. Obtener el score de tomas en el conjunto passengers.

```
127.0.0.1:6379> ZSCORE passengers tomas  
"2"
```

76. Extraer el elemento con menor score del conjunto passengers.

```
127.0.0.1:6379> ZPOPMIN passengers  
1) "ivan"  
2) "1"
```

77. Extraer el elemento con mayor score del conjunto passengers.

```
127.0.0.1:6379> ZPOPMAX passengers  
1) "alejandra"  
2) "6"
```

78. Eliminar del conjunto passengers al valor silvia.

```
127.0.0.1:6379> ZREM passengers silvia  
(integer) 1
```

```
127.0.0.1:6379> ZRANGE passengers 0 -1  
1) "tomas"  
2) "natalia"  
3) "federico"  
4) "luciana"  
5) "julian"
```

Sección 8 - Hashes

79. Agregar a un hash llamado user:cronos los siguientes campos:

"razon social"	"cronos s.a"
domicilio	"47 236 La Plata"
"telefono"	2215556677

```
127.0.0.1:6379> HSET user:cronos "razon social" "cronos s.a" domicilio "47 236 La Plata" telefono 2215556677  
(integer) 3
```

80. Agregar el campo mail con el valor info@cronos.com.ar al hash user:cronos.

```
127.0.0.1:6379> HSET user:cronos mail info@cronos.com.ar  
(integer) 1
```

81. Obtener todos los campos y valores de user:cronos.

```
127.0.0.1:6379> HGETALL user:cronos
1) "razon social"
2) "cronos s.a"
3) "domicilio"
4) "47 236 La Plata"
5) "telefono"
6) "2215556677"
7) "mail"
8) "info@cronos.com.ar"
```

82. Obtener únicamente el valor del campo mail de user:cronos.

```
127.0.0.1:6379> HGET user:cronos mail
"info@cronos.com.ar"
```

83. Eliminar el campo teléfono de user:cronos.

```
127.0.0.1:6379> HDEL user:cronos telefono
(integer) 1
127.0.0.1:6379> HGETALL user:cronos
1) "razon social"
2) "cronos s.a"
3) "domicilio"
4) "47 236 La Plata"
5) "mail"
6) "info@cronos.com.ar"
```

84. Obtener la cantidad de campos de user:cronos.

```
127.0.0.1:6379> HLEN user:cronos
(integer) 3
```

85. Obtener las claves (nombres de campos) de user:cronos.

```
127.0.0.1:6379> HKEYS user:cronos
1) "razon social"
2) "domicilio"
3) "mail"
```

86. Determinar si existe el campo cuil en user:cronos.

```
127.0.0.1:6379> HEXISTS user:cronos cuil
(integer) 0
```

87. Obtener todos los valores (sin los nombres de campos) de user:cronos.

```
127.0.0.1:6379> HVALS user:cronos
1) "cronos s.a"
2) "47 236 La Plata"
3) "info@cronos.com.ar"
```

88. Obtener la longitud del valor del campo mail de user:cronos.

```
127.0.0.1:6379> HSTRLEN user:cronos mail  
(integer) 18
```

Sección 9 - Geospatial

89. Agregar en un conjunto denominado cities las siguientes localidades con sus coordenadas (longitud, latitud):

Ciudad	Latitud	Longitud
Buenos Aires	-34.61315	-58.37723
Cordoba	-31.41350	-64.18105
Rosario	-32.94682	-60.63932
Mendoza	-32.89084	-68.82717
San Miguel de Tucuman	-26.82414	-65.22260
La Plata	-34.92145	-57.95453
Mar del Plata	-38.00042	-57.55620
Salta	-24.78590	-65.41166
Santa Fe	-31.64881	-60.70868
San Juan	-31.53750	-68.53639
Resistencia	-27.46056	-58.98389
Santiago del Estero	-27.79511	-64.26149
Posadas	-27.36708	-55.89608
San Salvador de Jujuy	-24.19457	-65.29712
Bahia Blanca	-38.71959	-62.27243
Parana	-31.73271	-60.52897

Atencion En Redis, los datos geoespaciales se almacenan como conjuntos ordenados (Sorted Sets). Las coordenadas se pasan en el orden longitud, latitud (no latitud, longitud).

Si le erré a algo, no me importa, soy más feliz sin saberlo

```
127.0.0.1:6379> GEOADD cities -58.37723 -34.61315 "Buenos Aires" -64.18105 -31.41350 Cordoba -60.63932 -32.94682 Rosario  
-68.82717 -32.89084 Mendoza -65.22260 -26.82414 "San Miguel de Tucuman" -57.95453 -34.92145 "La Plata" -57.55620 -38.00  
042 "Mar del Plata" -65.41166 -24.78590 Salta -60.70868 -31.64881 "Santa Fe" -68.53639 -31.53750 "San Juan" -58.98389 -2  
7.46056 Resistencia -64.26149 -27.79511 "Santiago del Estero" -55.89608 -27.36708 Posadas -65.29712 -24.19457 "San Salva  
dor de Jujuy" -62.27243 -38.71959 "Bahia Blanca" -60.52897 -31.73271 Parana  
(integer) 16
```

90. Obtener todos los miembros del conjunto cities.

```
127.0.0.1:6379> ZRANGE cities 0 -1  
1) "Mendoza"  
2) "San Juan"  
3) "Bahia Blanca"  
4) "Mar del Plata"  
5) "Buenos Aires"  
6) "La Plata"  
7) "Rosario"  
8) "Cordoba"  
9) "San Miguel de Tucuman"  
10) "Santiago del Estero"  
11) "Parana"  
12) "Santa Fe"  
13) "Resistencia"  
14) "Salta"  
15) "San Salvador de Jujuy"  
16) "Posadas"
```

91. Obtener las coordenadas almacenadas de Santa Fe.

```
127.0.0.1:6379> GEOPOS cities "Santa Fe"  
1) 1) "-60.708681643009186"  
2) "-31.64881101691541"
```

92. Obtener la distancia en kilómetros entre Buenos Aires y Córdoba.

```
127.0.0.1:6379> GEODIST cities "Buenos Aires" Cordoba KM  
"647.6303"
```

93. Obtener las ciudades que se encuentran en un radio de 100 km de la coordenada (-27.37, -55.9), incluyendo la distancia de cada una.

```
127.0.0.1:6379> GEOSEARCH cities FROMLONLAT -55.9 -27.37 BYRADIUS 100 KM WITHDIST  
1) 1) "Posadas"  
2) "0.5055"
```

94. Obtener las ciudades que se encuentran a menos de 700 km de Córdoba.

```
127.0.0.1:6379> GEOSEARCH cities FROMMEMBER Cordoba BYRADIUS 700 KM WITHDIST
1) 1) "Cordoba"
   2) "0.0000"
2) 1) "San Miguel de Tucuman"
   2) "520.3835"
3) 1) "Santiago del Estero"
   2) "402.5353"
4) 1) "Parana"
   2) "347.8771"
5) 1) "Santa Fe"
   2) "330.2202"
6) 1) "Resistencia"
   2) "668.2173"
7) 1) "Buenos Aires"
   2) "647.6303"
8) 1) "La Plata"
   2) "698.5502"
9) 1) "Rosario"
   2) "374.4698"
10) 1) "San Juan"
    2) "413.3545"
11) 1) "Mendoza"
    2) "467.3004"
```